

FIAP — Faculdade de Informática e Administração Paulista
Análise e Desenvolvimento de Sistemas

Domain Driven Design Using Java

PROJETO NORA

Backend Java — Documentação Técnica

Nome da solução

NORA

Equipe

Equipe Nora

Integrantes

Diego Costa Paulino — RM 566841

Guilherme Alves Dabul — RM 559901

Ryan Vieira de Aguiar Mazali — RM 567168

São Paulo — 2025

Sumário

Sumário	2
1. Objetivo e Escopo do Projeto	4
1.1. O que é o Projeto Nora	4
1.2. O que é este documento	4
1.3. O papel do backend Java	4
1.4. Tecnologias utilizadas	4
1.5. Escopo — o que está e o que não está incluído	5
Está dentro do escopo do backend Java	5
Está fora do escopo do backend Java	5
2. Principais Funcionalidades da Solução	6
2.1. Autenticação de colaboradores	6
2.2. Gestão de pessoas (leads)	6
2.3. Triagem dos casos	6
2.4. Gestão de pacientes	6
2.5. Gestão de dentistas voluntários	6
2.6. Encaminhamentos	6
2.7. Conversas e mensagens (omnichannel)	7
2.8. Acompanhamento do tratamento	7
2.9. Métricas e indicadores	7
2.10. Follow-up automático	7
2.11. Verificação de saúde (health check)	7
3. Métodos Lógicos do Sistema	8
3.1. Método definirElegibilidade — verifica se o adolescente pode entrar no programa	8
O que faz	8
A lógica por trás	8
Print do código	8
3.2. Método calcularPrioridade — define a urgência do atendimento	9
O que faz	9
A lógica por trás	9
Print do código	9
3.3. Método verificarDisponibilidade — confere se o dentista pode receber um novo caso ..	10
O que faz	10
A lógica por trás	10
Print do código	10
3.4. Método encerrarEncaminhamento — conclui um encaminhamento	10
O que faz	10
A lógica por trás	10
Print do código	11
4. Tabela de Endpoints da API REST	12
4.1. Endereço base da API	12
4.2. Endpoints por categoria	12
Autenticação	12
Verificação de saúde	12
Pessoas	12
Triagens	13
Pacientes	13
Dentistas	13
Encaminhamentos	14

Conversas e mensagens	14
Acompanhamento.....	14
Métricas	14
Follow-up	15
4.3. Significado dos códigos de status HTTP.....	15
5. Protótipo — Telas Implementadas	16
5.1. Tela de Login.....	16
5.2. Painel / Dashboard de Métricas.....	16
5.3. Lista de Leads / Pessoas.....	16
5.4. Tela de Triagens.....	17
5.5. Tela de Dentistas.....	17
5.6. Tela de Encaminhamentos	17
5.7. Omnichannel — Conversas e Chat.....	18
6. Fluxo Crítico — Aprovação de Triagem.....	19
6.1. Por que esse fluxo é especial	19
6.2. Passo a passo da aprovação.....	19
6.3. Como o dentista é selecionado.....	20
6.4. Efeito da aprovação no restante do sistema	20
7. Modelo de Entidade-Relacionamento (MER)	21
7.1. Diagrama do banco de dados.....	21
7.2. Tabelas do banco de dados.....	21
7.3. Principais relacionamentos.....	22
7.4. Destaque — a regra de exclusividade da conversa	23
8. Diagrama de Classes.....	24
8.1. Organização em camadas.....	24
8.2. Diagrama visual das classes	24
8.3. Classes de entidade e suas relações	25
8.4. Onde estão os métodos lógicos.....	25
9. Como Executar o Projeto	26
9.1. Pré-requisitos	26
9.2. Preparação do banco de dados.....	26
9.3. Configuração das variáveis de ambiente	26
9.4. Executando o projeto localmente.....	27
Modo de desenvolvimento	27
Gerar a versão de produção	27
Executar a versão empacotada.....	27
9.5. Verificando se o projeto está no ar	27
9.6. Testando a API.....	28
9.7. Hospedagem (deploy)	28
10. Considerações Finais.....	29
10.1. Resumo do que foi entregue.....	29
10.2. Decisões técnicas de destaque	29
10.3. Limitações conhecidas	29
10.4. Encerramento.....	29

1. Objetivo e Escopo do Projeto

1.1. O que é o Projeto Nora

O Projeto Nora é uma plataforma de gestão desenvolvida para apoiar a ONG Turma do Bem, organização que conecta adolescentes em situação de vulnerabilidade a dentistas voluntários. A plataforma cobre todo o caminho de atendimento: desde o primeiro contato do adolescente, passando pela avaliação do caso, até o encaminhamento para um dentista e o acompanhamento do tratamento.

O público atendido são adolescentes de 11 a 17 anos. Essa faixa de idade não é um detalhe qualquer: ela é uma regra de negócio central do sistema e influencia diretamente a forma como cada caso é avaliado, como será explicado na Seção 3.

1.2. O que é este documento

Esta documentação descreve especificamente o backend Java do Projeto Nora — a parte do sistema responsável por receber, processar, armazenar e disponibilizar os dados. O Projeto Nora é maior do que apenas o backend Java; ele também envolve um bot de atendimento, um frontend (a tela que o usuário vê) e serviços auxiliares. Este documento, porém, trata exclusivamente da aplicação Java.

O objetivo é que qualquer pessoa — incluindo o professor avaliador — consiga, lendo este documento, entender o que o sistema faz, como ele está organizado por dentro, quais são suas regras, e como colocá-lo para rodar.

1.3. O papel do backend Java

O backend Java é o núcleo do Projeto Nora. Ele funciona como um orquestrador: recebe as informações, aplica as regras, guarda tudo no banco de dados e entrega os dados prontos para quem precisar consultar. De forma resumida, o backend Java é responsável por:

- Receber dados de novos adolescentes (chamados de leads) que chegam por canais externos, como o bot de atendimento.
- Avaliar cada caso por meio de uma triagem, calculando se o adolescente é elegível ao programa e qual a urgência do atendimento.
- Transformar uma triagem aprovada em um paciente do programa.
- Encontrar um dentista voluntário com vaga disponível e criar o encaminhamento que liga o paciente a esse dentista.
- Manter o registro das conversas de atendimento (a chamada camada omnichannel).
- Disponibilizar métricas e indicadores que alimentam um painel (dashboard) de acompanhamento.
- Disparar lembretes automáticos de acompanhamento (follow-up) dos tratamentos em andamento.

1.4. Tecnologias utilizadas

O backend foi construído com tecnologias amplamente usadas no mercado e compatíveis com o conteúdo trabalhado em sala de aula. A tabela abaixo resume a stack do projeto:

Camada	Tecnologia escolhida
Linguagem de programação	Java 21
Framework da aplicação	Quarkus 3.34.6
Ferramenta de build	Maven
Banco de dados	Oracle (instância da FIAP)
Forma de persistência	JDBC manual — comandos SQL escritos à mão, sem JPA/Hibernate
Estilo da API	REST, troca de dados em formato JSON
Autenticação	Token opaco (UUID) gerado no login
Hospedagem (deploy)	Render
Verificação de saúde	Endpoint /q/health

A escolha por JDBC manual em vez de um framework de mapeamento automático (como JPA/Hibernate) foi proposital: ela mantém o código alinhado ao que foi ensinado nas aulas de Java e deixa visível, para quem lê o código, exatamente qual comando SQL está sendo executado em cada operação.

1.5. Escopo — o que está e o que não está incluído

Para que a avaliação seja justa, é importante deixar claro o limite do que este backend faz:

Está dentro do escopo do backend Java

- Cadastro e consulta de pessoas, pacientes, dentistas, triagens, encaminhamentos, conversas e mensagens.
- O cálculo automático de elegibilidade e prioridade de cada triagem.
- A aprovação de triagem, que é uma operação transacional (explicada na Seção 6).
- A seleção interna de dentistas disponíveis para encaminhamento.
- A geração de métricas para o painel.
- O disparo automático de follow-up por meio de um agendador (scheduler).

Está fora do escopo do backend Java

- A interface visual (telas) — desenvolvida separadamente no frontend.
- O bot de atendimento que conversa com o adolescente — desenvolvido em outra parte do projeto.
- A inteligência artificial de predição de urgência — quando não configurada, o backend usa como alternativa o valor de urgência recebido na própria requisição.

2. Principais Funcionalidades da Solução

Esta seção apresenta, de forma organizada, tudo o que o backend Java do Projeto Nora é capaz de fazer. Cada funcionalidade é descrita de maneira objetiva, do ponto de vista de quem usa o sistema.

2.1. Autenticação de colaboradores

Antes de usar o sistema, o colaborador da ONG precisa fazer login informando seu e-mail (ou nome de login) e senha. Quando as credenciais estão corretas, o sistema devolve um token — uma espécie de “crachá digital” que identifica aquele colaborador. A partir daí, todas as demais operações exigem esse token. Sem ele, o sistema recusa o acesso.

2.2. Gestão de pessoas (leads)

Uma pessoa é o adolescente que entra em contato buscando atendimento — também chamado de lead enquanto ainda não foi aprovado. O sistema permite cadastrar uma nova pessoa, listar todas as pessoas cadastradas, consultar uma pessoa específica e atualizar seus dados.

2.3. Triagem dos casos

A triagem é a avaliação do caso de cada adolescente. Ao criar uma triagem, o sistema faz dois cálculos automáticos importantes:

- **Elegibilidade:** verifica se a idade do adolescente está dentro da faixa de 11 a 17 anos.
- **Prioridade:** classifica a urgência do atendimento (baixa, média, alta ou urgente) com base no nível de urgência informado.

É possível listar todas as triagens, consultar uma específica e atualizar seu status. A triagem também pode ser aprovada, o que aciona o fluxo mais importante do sistema (detalhado na Seção 6).

2.4. Gestão de pacientes

Quando uma triagem é aprovada, a pessoa correspondente passa a ser um paciente do programa. O sistema permite listar todos os pacientes e consultar um paciente específico, sempre trazendo junto os dados da pessoa vinculada.

2.5. Gestão de dentistas voluntários

Os dentistas são os profissionais voluntários que atendem os pacientes. O sistema permite cadastrar, listar, consultar e atualizar dentistas. Existe ainda uma consulta especial — dentistas disponíveis — que retorna apenas os dentistas ativos que ainda têm vagas no mês, respeitando sua capacidade mensal de atendimento.

2.6. Encaminhamentos

Um encaminhamento é o vínculo entre um paciente e um dentista. Ele pode ser criado automaticamente (durante a aprovação de uma triagem) ou manualmente. O sistema permite listar, consultar e atualizar encaminhamentos, incluindo o registro de sua conclusão.

2.7. Conversas e mensagens (omnichannel)

O sistema mantém o registro das conversas de atendimento. Cada conversa pertence a um contexto — pode estar ligada a uma pessoa em cadastro, a um paciente em acompanhamento, ou a um dentista. Dentro de cada conversa ficam armazenadas as mensagens trocadas. O sistema permite listar conversas (com filtros), consultar uma conversa, criar conversas e registrar mensagens.

2.8. Acompanhamento do tratamento

Para cada encaminhamento, o sistema registra eventos de acompanhamento — por exemplo, a primeira consulta, uma atualização do tratamento, ou a conclusão. Esses eventos formam o histórico do tratamento daquele paciente.

2.9. Métricas e indicadores

O backend disponibiliza um conjunto de consultas de métricas que alimentam o painel de acompanhamento da ONG: total de leads, total de aprovados, taxa de aprovação, distribuição de triagens por status, encaminhamentos por prioridade, leads por canal de origem, leads por mês e distribuição de leads por região (bairro).

2.10. Follow-up automático

O sistema possui um agendador que roda diariamente e identifica os encaminhamentos cujo prazo de acompanhamento chegou. Para cada um, ele dispara um lembrete e registra um evento de acompanhamento. Há também a opção de disparar esse processo manualmente, sem esperar o horário agendado.

2.11. Verificação de saúde (health check)

O backend expõe um endpoint de saúde que permite verificar, a qualquer momento, se a aplicação está no ar e respondendo. Esse recurso é utilizado pela plataforma de hospedagem para monitorar o serviço.

3. Métodos Lógicos do Sistema

Esta seção apresenta os quatro métodos lógicos exigidos pela atividade. Cada um deles representa uma regra de negócio do Projeto Nora e foi implementado dentro de uma entidade de domínio — ou seja, dentro da própria classe que representa o conceito do mundo real (Triagem, Dentista e Encaminhamento).

Colocar a regra de negócio dentro da entidade é uma decisão de organização: a regra fica perto do dado que ela governa, fácil de localizar e de testar. Para cada método, o documento traz a explicação da lógica e o print do código real implementado no projeto.

Por que dentro das entidades?

No Projeto Nora, as classes da pasta entities representam os conceitos do domínio (Triagem, Dentista, Encaminhamento, etc.). Quando uma regra depende apenas dos dados daquele conceito, ela mora na própria entidade. Isso evita espalhar regras por várias partes do sistema e torna o código mais previsível.

3.1. Método definirElegibilidade — verifica se o adolescente pode entrar no programa

Classe: Triagem | **Retorno:** texto (“elegível” ou “inelegível”)

O que faz

Este método recebe a idade do adolescente e decide se ele é elegível ao programa. A regra do Projeto Nora atende apenas adolescentes de 11 a 17 anos. Portanto:

- **Se a idade estiver entre 11 e 17 anos (inclusive):** o método devolve “elegível”.
- **Se a idade estiver fora dessa faixa:** o método devolve “inelegível”.

A lógica por trás

A condição usa o operador lógico E (escrito como && em Java). Isso significa que as duas comparações precisam ser verdadeiras ao mesmo tempo: a idade precisa ser maior ou igual a 11 E menor ou igual a 17. Se qualquer uma das duas falhar, o adolescente é considerado inelegível. Um detalhe importante: a fonte da verdade é este cálculo do backend — mesmo que a requisição tente informar outra elegibilidade, é o resultado deste método que vale.

Print do código

```
1 // Triagem.java – entidade de domínio
2 // Faixa de elegibilidade do Projeto Nora: adolescentes de 11 a 17 anos.
3 public String definirElegibilidade(int idade) {
4     if (idade >= 11 && idade <= 17) return "elegível";
5     return "inelegível";
6 }
```


3.2. Método calcularPrioridade — define a urgência do atendimento

Classe: Triagem | **Retorno:** texto (“urgente”, “alta”, “media” ou “baixa”)

O que faz

Este método recebe o nível de urgência do caso (um número, normalmente entre 0 e 5) e o converte em uma classificação de prioridade que faz sentido para o atendimento. Quanto maior o número, mais urgente é o caso.

A lógica por trás

O método usa uma sequência de verificações em ordem decrescente — do mais grave para o menos grave. A ordem aqui é fundamental: o método testa primeiro a faixa mais alta e, assim que encontra a primeira condição verdadeira, devolve o resultado e encerra. As faixas são:

Nível de urgência	Prioridade resultante
Maior ou igual a 4.0	urgente
Maior ou igual a 3.0 (e menor que 4.0)	alta
Maior ou igual a 2.0 (e menor que 3.0)	media
Menor que 2.0	baixa

Como cada verificação só é alcançada se a anterior falhou, não é necessário escrever o limite superior de cada faixa — ele fica implícito. Por exemplo: se o código chegou à verificação do 3.0, é porque o valor já não era 4.0 ou mais.

Print do código

```
1 // Triagem.java – entidade de domínio
2 // Prioridade calculada a partir do nível de urgencia retornado
3 // pelo servico de predicao (nível de urgencia da IA).
4 public String calcularPrioridade(double nivelIA) {
5     if (nivelIA >= 4.0) return "urgente";
6     if (nivelIA >= 3.0) return "alta";
7     if (nivelIA >= 2.0) return "media";
8     return "baixa";
9 }
```

3.3. Método verificarDisponibilidade — confere se o dentista pode receber um novo caso

Classe: Dentista | **Retorno:** verdadeiro ou falso (disponível ou não)

O que faz

Este método decide se um dentista está apto a receber mais um encaminhamento. Para um dentista estar disponível, duas condições precisam ser atendidas ao mesmo tempo:

- **Ainda ter vaga no mês:** a quantidade de encaminhamentos ativos precisa ser menor que sua capacidade mensal.
- **Estar ativo:** o status do dentista precisa ser “ativo” (dentistas inativos ou suspensos não recebem casos).

A lógica por trás

Assim como no método de elegibilidade, este método usa o operador E (&&): ambas as condições precisam ser verdadeiras. Se o dentista estiver com a agenda cheia, OU se ele não estiver ativo, o método devolve falso.

Há um detalhe técnico interessante: a quantidade de encaminhamentos ativos não é um campo guardado na tabela do dentista. Esse número é calculado em outro ponto do sistema (contando os encaminhamentos ativos daquele dentista no banco) e é passado para o método como parâmetro. Isso mantém o dado sempre atualizado, sem risco de ficar “desatualizado” na tabela.

Print do código

```
1 // Dentista.java – entidade de domínio
2 // 'ativos' e calculado externamente (COUNT de encaminhamentos ativos)
3 // – nao existe coluna ATIVOS na tabela.
4 public boolean verificarDisponibilidade(int ativos) {
5     return ativos < this.capMensal && "ativo".equals(this.sttsDent);
6 }
```

3.4. Método encerrarEncaminhamento — conclui um encaminhamento

Classe: Encaminhamento | **Retorno:** nenhum (apenas altera o estado do objeto)

O que faz

Este método marca um encaminhamento como concluído. Ao ser chamado, ele altera o status do encaminhamento para “concluído”, registrando que aquele atendimento chegou ao fim.

A lógica por trás

É o mais simples dos quatro métodos, e isso é intencional. Diferente dos anteriores, ele não recebe nenhum parâmetro e não devolve nenhum valor — ele apenas modifica o próprio objeto. A palavra this no código indica que a alteração acontece no próprio encaminhamento sobre o qual o método foi chamado.

Vale registrar uma decisão de projeto: este método cuida apenas do status do encaminhamento. A eventual atualização do status do tratamento do paciente é responsabilidade do fluxo de acompanhamento, e não deste método. Manter cada método com uma responsabilidade clara e única é uma boa prática que torna o sistema mais fácil de entender e manter.

Print do código

```
1 // Encaminhamento.java – entidade de domínio
2 // Encerra o encaminhamento, alterando seu status para "concluído".
3 public void encerrarEncaminhamento() {
4     this.sttsEncam = "concluído";
5 }
```

Resumo dos quatro métodos lógicos

definirElegibilidade (Triagem) — verifica se a idade está na faixa de 11 a 17 anos.

calcularPrioridade (Triagem) — converte o nível de urgência em baixa, média, alta ou urgente.

verificarDisponibilidade (Dentista) — confere se o dentista tem vaga e está ativo.

encerrarEncaminhamento (Encaminhamento) — marca o encaminhamento como concluído.

4. Tabela de Endpoints da API REST

Esta seção lista todos os endpoints (pontos de acesso) da API REST do backend Java. Cada endpoint é uma “porta de entrada” do sistema: o frontend e os demais consumidores fazem requisições a esses caminhos para ler ou gravar informações.

Cada linha das tabelas abaixo informa o verbo HTTP (a intenção da requisição), a URI (o caminho do recurso), uma breve descrição e os códigos de status HTTP que aquele endpoint pode devolver.

Como ler os verbos HTTP

GET — solicita uma informação, sem alterar nada (consulta).

POST — envia uma informação nova para o sistema criar um recurso.

PUT — envia uma informação para atualizar um recurso que já existe.

O trecho {id} nas URIs é um espaço reservado: ele deve ser substituído pelo número identificador do recurso desejado (por exemplo, /pessoas/5).

4.1. Endereço base da API

Em ambiente de desenvolvimento (na máquina do desenvolvedor), a API responde no endereço `http://localhost:10000`. Em produção, responde no endereço fornecido pela plataforma Render após o deploy. As URIs das tabelas abaixo devem ser acrescentadas a esse endereço base.

4.2. Endpoints por categoria

Autenticação

Verbo	URI	Descrição	Status esperados
POST	/auth/login	Autentica o colaborador e devolve o token de acesso.	200, 400, 401, 500

Verificação de saúde

Verbo	URI	Descrição	Status esperados
GET	/q/health	Verifica se a aplicação está no ar (não exige token).	200

Pessoas

Verbo	URI	Descrição	Status esperados
GET	/pessoas	Lista todas as pessoas cadastradas.	200, 500
POST	/pessoas	Cadastra uma nova pessoa (lead).	201, 400, 500

Verbo	URI	Descrição	Status esperados
GET	/pessoas/{id}	Busca uma pessoa pelo seu identificador.	200, 404, 500
PUT	/pessoas/{id}	Atualiza os dados de uma pessoa existente.	200, 400, 404, 500

Triagens

Verbo	URI	Descrição	Status esperados
GET	/triagens	Lista todas as triagens.	200, 500
POST	/triagens	Cria uma triagem (calcula elegibilidade e prioridade).	201, 400, 422, 500
GET	/triagens/{id}	Busca uma triagem pelo seu identificador.	200, 404, 500
PUT	/triagens/{id}	Atualiza o status e a decisão de uma triagem.	200, 400, 404, 500
POST	/triagens/{id}/aprovar	Aprova a triagem em uma transação atômica (fluxo crítico).	201, 404, 409, 422, 500

Pacientes

Verbo	URI	Descrição	Status esperados
GET	/pacientes	Lista todos os pacientes aprovados.	200, 500
GET	/pacientes/{id}	Busca um paciente pelo identificador da pessoa.	200, 404, 500

Dentistas

Verbo	URI	Descrição	Status esperados
GET	/dentistas	Lista todos os dentistas cadastrados.	200, 500
POST	/dentistas	Cadastra um novo dentista voluntário.	201, 400, 500
GET	/dentistas/disponiveis	Lista dentistas ativos com vagas disponíveis.	200, 500
GET	/dentistas/{id}	Busca um dentista pelo seu identificador.	200, 404, 500
PUT	/dentistas/{id}	Atualiza os dados de um dentista existente.	200, 400, 404, 500

Encaminhamentos

Verbo	URI	Descrição	Status esperados
GET	/encaminhamentos	Lista todos os encaminhamentos.	200, 500
POST	/encaminhamentos	Cria um encaminhamento manual.	201, 400, 404, 422, 500
GET	/encaminhamentos/{id}	Busca um encaminhamento pelo identificador.	200, 404, 500
PUT	/encaminhamentos/{id}	Atualiza o status e a observação do encaminhamento.	200, 400, 404, 500
GET	/encaminhamentos/{id}/eventos	Lista os eventos de acompanhamento do encaminhamento.	200, 404

Conversas e mensagens

Verbo	URI	Descrição	Status esperados
GET	/conversas	Lista as conversas (aceita filtros por contexto e status).	200, 500
POST	/conversas	Cria uma conversa (reaproveita conversa ativa já existente).	200, 201, 404, 422, 500
GET	/conversas/{id}	Busca uma conversa pelo seu identificador.	200, 404, 500
GET	/conversas/{id}/mensagens	Lista as mensagens de uma conversa.	200, 404, 500
POST	/mensagens	Registra uma nova mensagem em uma conversa.	201, 404, 422, 500

Acompanhamento

Verbo	URI	Descrição	Status esperados
POST	/acomp_evento	Registra um evento de acompanhamento.	201, 404, 422, 500

Métricas

Verbo	URI	Descrição	Status esperados
GET	/metricas/resumo	Totais e taxas consolidadas da operação.	200, 500
GET	/metricas/triagens-por-status	Distribuição de triagens por status.	200, 500

Verbo	URI	Descrição	Status esperados
GET	/metricas/encaminhamentos-por-prioridade	Distribuição de encaminhamentos por prioridade.	200, 500
GET	/metricas/leads-por-canal	Quantidade de leads por canal de origem.	200, 500
GET	/metricas/leads-por-mes	Quantidade de leads por mês de cadastro.	200, 500
GET	/metricas/regioes	Quantidade de leads por região (bairro).	200, 500

Follow-up

Verbo	URI	Descrição	Status esperados
POST	/follow-up/executar	Dispara o processo de follow-up manualmente.	200, 500

4.3. Significado dos códigos de status HTTP

Os códigos de status são a forma que a API tem de informar como a requisição terminou. A tabela abaixo explica cada código utilizado no Projeto Nora:

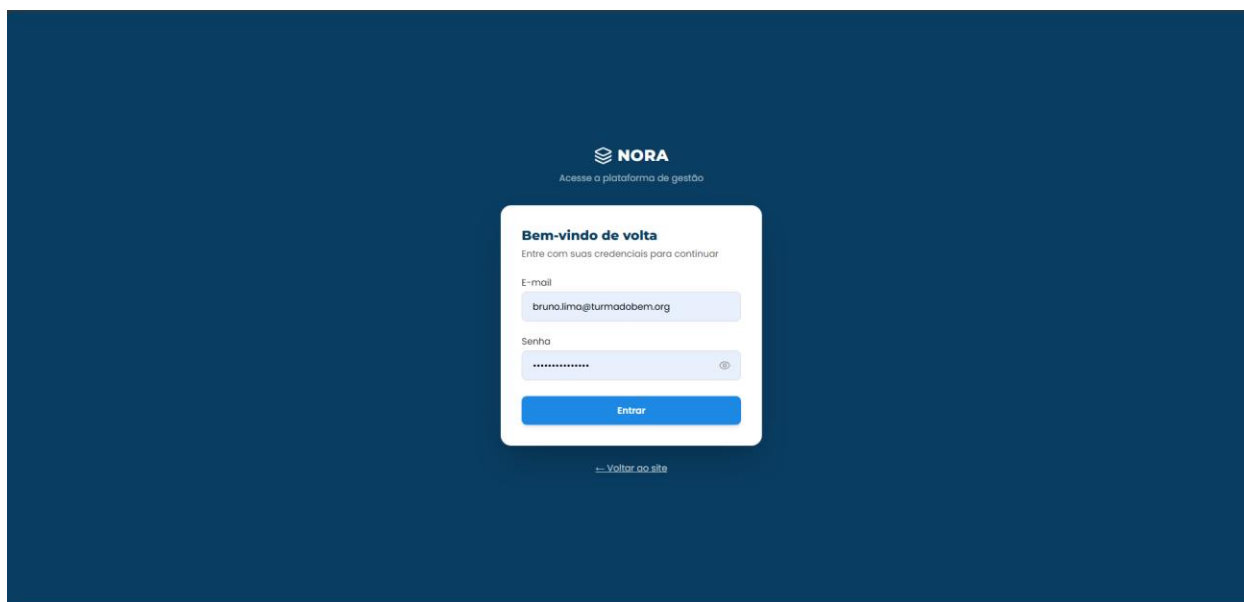
Código	Significado no Projeto Nora
200	Sucesso. A requisição foi atendida (usado em consultas e atualizações).
201	Criado. Um novo recurso foi criado com sucesso.
400	Requisição inválida. Falta um campo obrigatório ou um dado está malformatado.
401	Não autorizado. O token de acesso está ausente ou é inválido.
404	Não encontrado. O recurso solicitado não existe.
409	Conflito. A operação contraria o estado atual (ex.: aprovar uma triagem já aprovada).
422	Regra de negócio violada (ex.: nenhum dentista disponível para o encaminhamento).
500	Erro interno. Ocorreu uma falha inesperada no servidor.

5. Protótipo — Telas Implementadas

Esta seção apresenta o protótipo do sistema por meio de capturas de tela (prints) reais da aplicação em funcionamento. As telas correspondem ao frontend do Projeto Nora, que consome a API REST descrita na Seção 4.

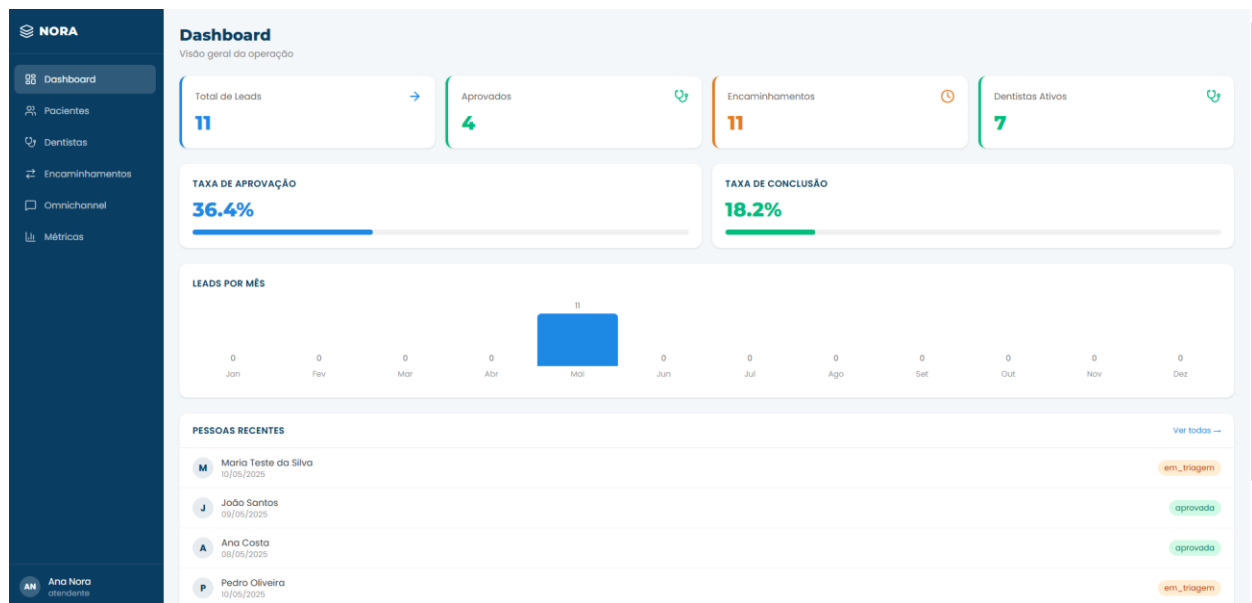
5.1. Tela de Login

Tela inicial do sistema, onde o colaborador da ONG informa suas credenciais para acessar a plataforma. Consome o endpoint POST /auth/login.



5.2. Painel / Dashboard de Métricas

Tela que apresenta os indicadores consolidados da operação: total de leads, total de aprovados, taxa de aprovação, gráficos de triagens por status, encaminhamentos por prioridade, leads por canal e por mês. Consome os endpoints da categoria /metrics.



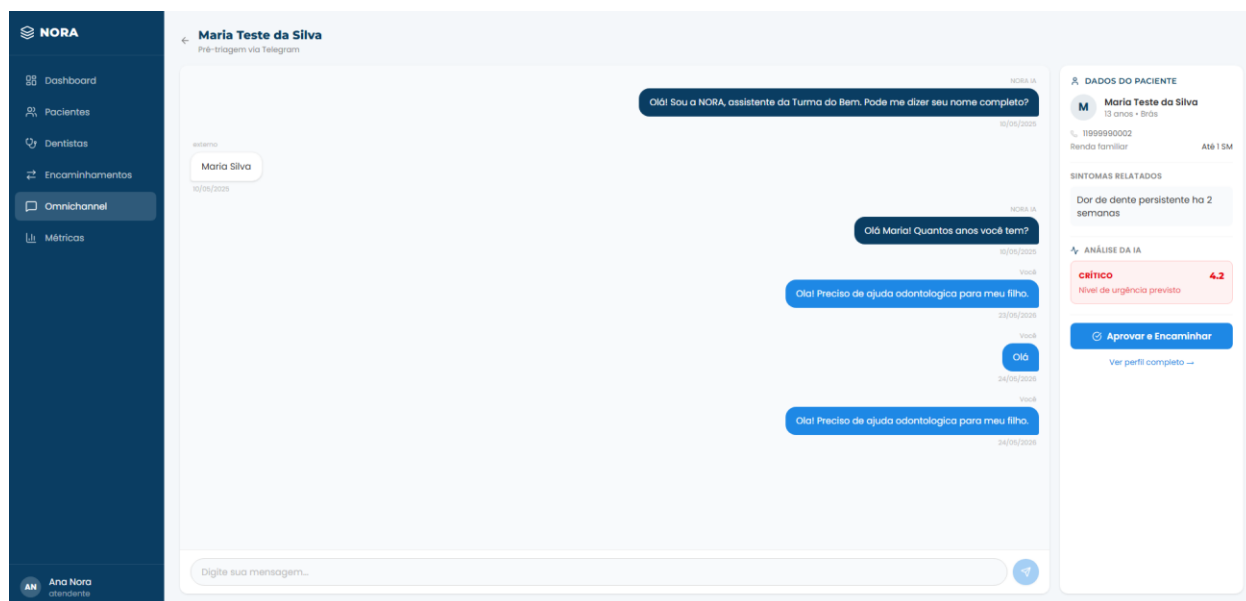
5.3. Lista de Leads / Pessoas

Tela que lista todas as pessoas (leads) cadastradas no sistema. Consome o endpoint GET /pessoas.



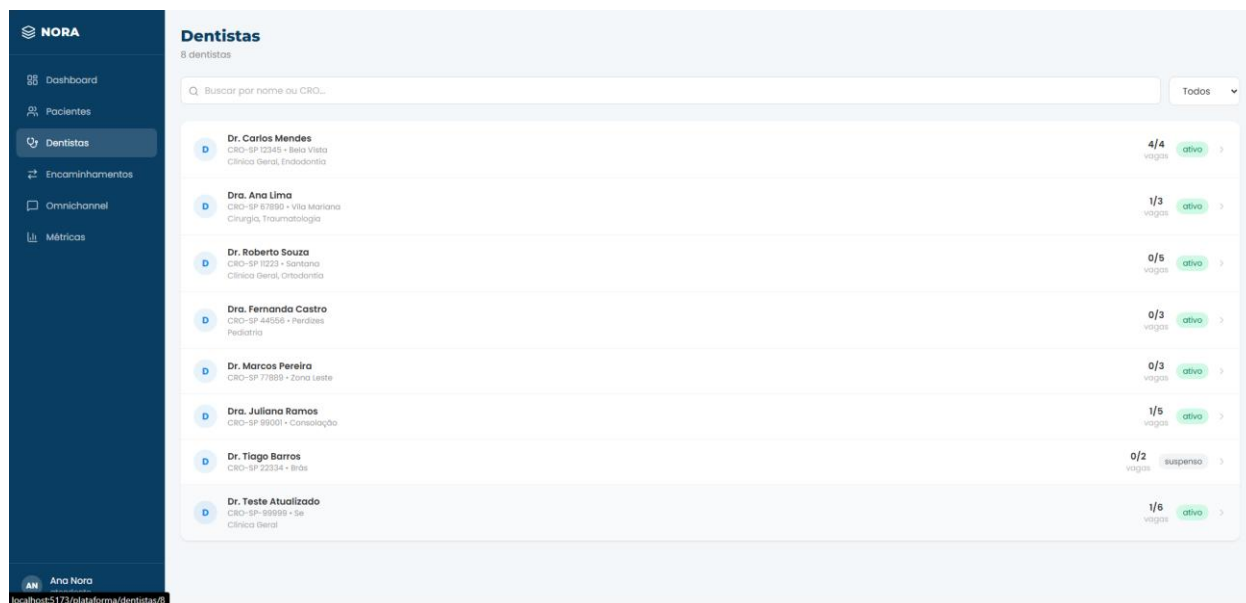
5.4. Tela de Triagens

Tela onde os casos são avaliados. Mostra a lista de triagens e permite criar uma nova triagem e aprovar triagens existentes. Consome os endpoints GET /triagens, POST /triagens e POST /triagens/{id}/aprovar.



5.5. Tela de Dentistas

Tela que lista os dentistas voluntários cadastrados, com seus dados, capacidade mensal e status. Consome o endpoint GET /dentistas.



5.6. Tela de Encaminhamentos

Tela que lista os encaminhamentos, mostrando o vínculo entre paciente e dentista, a prioridade e o status. Consome o endpoint GET /encaminhamentos.

Encaminhamentos				Todos os status
João Santos	→ Dr. Carlos Mendes	08/05/2025	media concluido	
Ana Costa	→ Dra. Ana Lima	08/05/2025	urgente ativa	
Lucas Ferreira	→ Dr. Carlos Mendes	07/05/2025	baixa ativa	
Rafael Costa	→ Dr. Roberto Souza	25/04/2025	media concluido	
Maria Teste da Silva	→ Dra. Fernanda Castro	15/04/2025	alta cancelado	
Pedro Oliveira	→ Dra. Ana Lima	10/03/2025	media reencaminhado	
Beatriz Alves	→ Dra. Juliana Ramos	01/05/2025	alta ativa	
Maria Teste da Silva	→ Dr. Carlos Mendes	23/05/2026	baixa ativa	
João Santos	→ Dr. Carlos Mendes	24/05/2026	alta ativa	
Maria Teste da Silva	→ Dr. Carlos Mendes	24/05/2026	baixa ativa	
João Santos	→ Dr. Teste Atualizado	24/05/2026	media ativa	

5.7. Omnichannel — Conversas e Chat

Tela de atendimento que mostra as conversas em andamento e permite visualizar as mensagens trocadas. Consome os endpoints GET /conversas e GET /conversas/{id}/mensagens.

Omnichannel		
Central de conversas e acompanhamento		
4	4	2
Pré-triagem	Pacientes	Dentistas
Todos Pré-triagem Follow-up		
M	Maria Teste da Silva	24/05/2026
Olá! Preciso de ajuda odontológica para meu filho.		
Telegram: pré-triagem aberta		
J	João Santos	
Sem mensagens		
Telegram: pré-triagem aberta		
C	Dr. Carlos Mendes	10/05/2025
Perfeito Dr. Carlos! Registro atualizado. Obrigado pela dedicação!		
Telegram: follow-up aberta		
A	Dra. Ana Lima	
Sem mensagens		
Telegram: follow-up aberta		
L	Lucas Ferreira	
Sem mensagens		
Telegram: pré-triagem encerrada		
A	Ana Costa	
Sem mensagens		
Telegram: follow-up aberta		
B	Beatriz Alves	
Sem mensagens		
Telegram: pré-triagem encerrada		

6. Fluxo Crítico — Aprovação de Triagem

Entre todas as operações do backend, a aprovação de triagem é a mais importante e a mais delicada. Ela é acionada pelo endpoint POST `/triagens/{id}/aprovar` e merece uma seção própria porque, diferente de um cadastro simples, ela altera vários registros do banco de uma só vez.

6.1. Por que esse fluxo é especial

Aprovar uma triagem não é apenas marcar a triagem como aprovada. Esse único clique dispara uma sequência de mudanças encadeadas: a pessoa vira paciente, a conversa muda de contexto, um dentista é selecionado e um encaminhamento é criado. Se qualquer uma dessas etapas falhar no meio do caminho, o sistema ficaria em um estado inconsistente — por exemplo, com um paciente criado, mas sem o encaminhamento correspondente.

Para evitar isso, o fluxo é executado dentro de uma transação atômica. “Atômica” significa “tudo ou nada”: ou todas as etapas são concluídas com sucesso e gravadas juntas, ou nenhuma delas é gravada. Se ocorrer uma falha, o sistema desfaz tudo o que havia feito (operação chamada de rollback) e o banco volta exatamente ao estado anterior.

O que é uma transação atômica

Imagine uma transferência bancária: o dinheiro precisa sair de uma conta E entrar na outra. Não pode sair de uma e não entrar na outra. A transação garante que as duas coisas aconteçam juntas, ou nenhuma aconteça. No Projeto Nora, a aprovação de triagem usa exatamente esse princípio para nunca deixar o banco “pela metade”.

6.2. Passo a passo da aprovação

Quando o endpoint de aprovação é chamado, o backend executa, em ordem, os seguintes passos — todos dentro da mesma transação:

1. Busca a triagem pelo identificador informado.
2. Verifica se a triagem existe. Se não existir, devolve o status **404** e encerra.
3. Verifica se a triagem ainda pode ser aprovada. Se já estiver aprovada ou encerrada, devolve o status **409** (conflito) e encerra.
4. Verifica se a pessoa daquela triagem já não possui um paciente cadastrado, evitando duplicidade.
5. Procura um dentista disponível no banco de dados. Se não houver nenhum, devolve o status **422** e encerra.
6. Atualiza a triagem: o status passa a ser “aprovada” e a decisão passa a ser “aprovado”.
7. Atualiza a pessoa: o status dela passa a ser “aprovada”.
8. Cria o registro do paciente, vinculado àquela pessoa, com a data de aprovação.
9. Se existir uma conversa vinculada à pessoa, migra essa conversa para o contexto “acomp_paciente” (acompanhamento de paciente).
10. Cria o encaminhamento, ligando o paciente ao dentista selecionado, com a previsão de follow-up definida para 15 dias à frente.

11. Cria um evento de acompanhamento registrando que o encaminhamento foi gerado.
12. Confirma a transação (commit). Se todos os passos deram certo, todas as mudanças são gravadas de uma vez.

Se ocorrer um erro em qualquer um dos passos acima, o backend executa o rollback: todas as alterações são desfeitas e o banco permanece como estava antes da tentativa de aprovação. O sistema então devolve um código de erro apropriado, explicando o motivo da falha.

6.3. Como o dentista é selecionado

No Projeto Nora, a escolha do dentista durante a aprovação é feita internamente, consultando o próprio banco de dados Oracle. O sistema busca o primeiro dentista que esteja ativo e que ainda tenha vaga no mês (ou seja, cuja quantidade de encaminhamentos ativos seja menor que sua capacidade mensal). Não há, nesse fluxo, chamada a um serviço externo de localização geográfica — por isso, o campo de distância em quilômetros fica vazio nos encaminhamentos criados automaticamente.

6.4. Efeito da aprovação no restante do sistema

Depois que uma triagem é aprovada com sucesso, o reflexo aparece em várias telas:

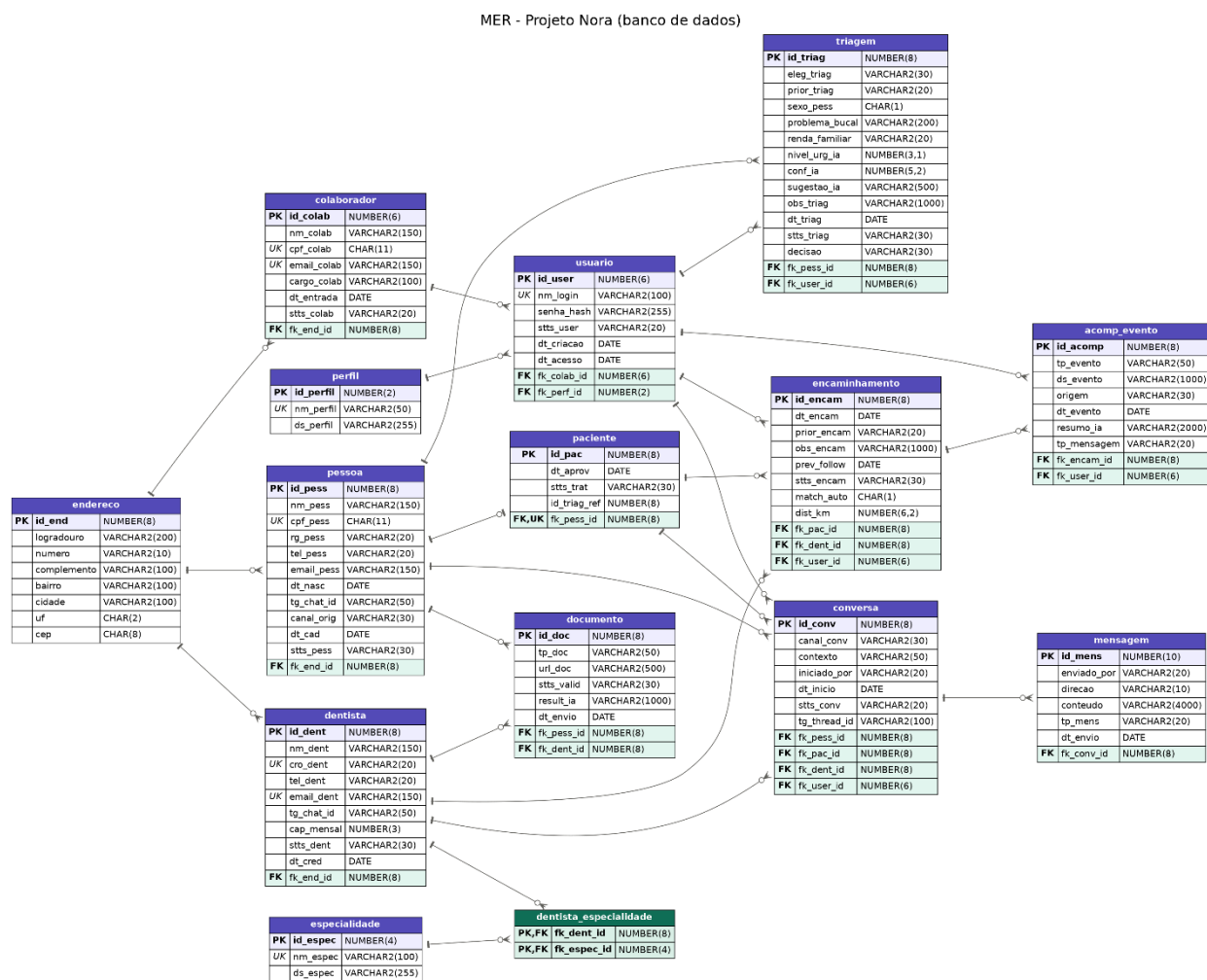
- O caso deixa de aparecer na lista de leads em pré-triagem.
- O caso passa a aparecer na camada de acompanhamento (casos ativos).
- A pessoa agora aparece na tela de pacientes.
- Um novo encaminhamento aparece na tela de encaminhamentos.

7. Modelo de Entidade-Relacionamento (MER)

O Modelo de Entidade-Relacionamento descreve como os dados do Projeto Nora estão organizados no banco de dados Oracle. O banco é composto por 15 tabelas, cada uma representando um conceito do sistema, ligadas entre si por relacionamentos.

7.1. Diagrama do banco de dados

O diagrama visual completo do MER, contendo todas as tabelas, atributos, chaves primárias, chaves estrangeiras e os relacionamentos, deve ser inserido no espaço abaixo. Ele pode ser gerado a partir do script de criação do banco (arquivo setup_oracle.sql) usando uma ferramenta como o SQL Developer Data Modeler, o dbdiagram.io ou ferramenta equivalente.



7.2. Tabelas do banco de dados

A tabela abaixo descreve cada uma das 15 tabelas do banco, com sua função e a quantidade aproximada de colunas:

Tabela	Função	Nº de colunas
endereco	Endereços usados por pessoas, colaboradores e dentistas.	8
perfil	Perfis de acesso dos colaboradores (atendente, coordenador, etc.).	3
colaborador	Funcionários e voluntários internos da ONG.	8
usuario	Credenciais de login, vinculadas a um colaborador e a um perfil.	8
pessoa	Adolescentes cadastrados (leads); raiz do fluxo de atendimento.	12
dentista	Dentistas voluntários que recebem os encaminhamentos.	10
especialidade	Especialidades odontológicas (clínica geral, ortodontia, etc.).	3
dentista_especialidade	Tabela de ligação entre dentista e especialidade.	2
triagem	Avaliação de cada caso: elegibilidade, prioridade e dados da IA.	15
paciente	Pessoa aprovada, apta a receber encaminhamento.	5
encaminhamento	Vínculo entre um paciente e um dentista.	11
acompanhamento_evento	Eventos de acompanhamento de um encaminhamento.	9
conversa	Conversas de atendimento (omnichannel).	11
mensagem	Mensagens trocadas dentro de uma conversa.	7
documento	Documentos enviados por pessoas ou dentistas.	8

7.3. Principais relacionamentos

Os relacionamentos definem como as tabelas se conectam. No banco, essas conexões são feitas por chaves estrangeiras — colunas que apontam para o identificador de outra tabela. A tabela abaixo lista os principais relacionamentos:

Tabela	Referencia	Descrição do relacionamento
pessoa	endereco	Cada pessoa pode ter um endereço (muitos para um).
colaborador	endereco	Cada colaborador pode ter um endereço (muitos para um).
dentista	endereco	Cada dentista pode ter um endereço (muitos para um).
usuario	colaborador	Cada usuário pertence a um colaborador (muitos para um).

Tabela	Referencia	Descrição do relacionamento
usuario	perfil	Cada usuário possui um perfil de acesso (muitos para um).
triagem	pessoa	Cada triagem avalia uma pessoa (muitos para um).
paciente	pessoa	Cada paciente corresponde a uma única pessoa (um para um).
encaminhamento	paciente	Cada encaminhamento pertence a um paciente (muitos para um).
encaminhamento	dentista	Cada encaminhamento aponta para um dentista (muitos para um).
acomp_evento	encaminhamento	Cada evento pertence a um encaminhamento (muitos para um).
conversa	pessoa / paciente / dentista	Cada conversa liga-se a apenas um dos três (regra de exclusividade).
mensagem	conversa	Cada mensagem pertence a uma conversa (muitos para um).
dentista_especialidade	dentista e especialidade	Liga dentistas a especialidades (muitos para muitos).

7.4. Destaque — a regra de exclusividade da conversa

Um ponto que merece atenção no modelo é a tabela conversa. Ela possui três possíveis vínculos: pode estar ligada a uma pessoa, a um paciente ou a um dentista. A regra de negócio determina que apenas um desses três vínculos pode estar preenchido por conversa — nunca dois ao mesmo tempo.

Essa regra não fica apenas no código: ela é garantida pelo próprio banco de dados, por meio de uma restrição (constraint). Assim, mesmo que o código tentasse gravar uma conversa com dois vínculos, o Oracle recusaria a operação. Isso garante que o contexto da conversa (cadastro, acompanhamento de paciente ou acompanhamento de dentista) seja sempre coerente.

8. Diagrama de Classes

O Diagrama de Classes mostra como o código do backend Java está organizado. Enquanto o MER (Seção 7) descreve a estrutura do banco de dados, o diagrama de classes descreve a estrutura do programa — quais classes existem, quais dados elas guardam e como se relacionam.

8.1. Organização em camadas

O backend é organizado em camadas, cada uma com uma responsabilidade clara. Essa separação é uma boa prática de arquitetura de software: ela mantém cada parte do código focada em uma única tarefa, facilitando o entendimento e a manutenção.

Camada (pacote)	Responsabilidade
entities	Representam os conceitos do domínio (Triagem, Dentista, etc.) e guardam as regras de negócio.
dao	Acesso ao banco de dados — executam os comandos SQL de inserção, consulta e atualização.
bo	Camada de regras de negócio compostas, situada entre os endpoints e o acesso a dados.
services	Orquestram fluxos mais complexos, como a aprovação de triagem e o follow-up.
resources	Endpoints REST — recebem as requisições HTTP e devolvem as respostas.
dto	Objetos de transporte — definem o formato dos dados de entrada e de saída da API.
mapper	Convertem entidades em DTOs e vice-versa.
conexaoes	Centralizam a abertura de conexões com o banco de dados Oracle.
filters	Filtros que tratam CORS e a verificação do token de autenticação.
exceptions	Definem os erros de regra de negócio do sistema.

8.2. Diagrama visual das classes

O diagrama visual completo de classes — mostrando os atributos, os métodos e as associações, heranças e demais relações entre as classes — deve ser inserido no espaço abaixo. Ele pode ser gerado a partir do código-fonte usando uma ferramenta de modelagem UML, como o Astah, o draw.io, ou o gerador de diagramas da própria IDE.

A tabela abaixo lista as classes de entidade do domínio, sua função e as classes com as quais cada uma se relaciona:

Classe	Função	Relaciona-se com
Pessoa	Adolescente cadastrado (lead); raiz do fluxo.	Endereco
Triagem	Avaliação do caso; contém regras de elegibilidade e prioridade.	Pessoa, Usuario
Paciente	Pessoa aprovada, apta a encaminhamento.	Pessoa
Dentista	Voluntário que recebe encaminhamentos; contém regra de disponibilidade.	Endereco
Encaminhamento	Vínculo paciente-dentista; contém regra de encerramento.	Paciente, Dentista, Usuario
Conversa	Conversa de atendimento (omnichannel).	Pessoa, Paciente, Dentista, Usuario
Mensagem	Mensagem dentro de uma conversa.	Conversa
AcompEvento	Evento de acompanhamento de um encaminhamento.	Encaminhamento, Usuario
Endereco	Endereço utilizado por pessoas, colaboradores e dentistas.	—
Colaborador	Funcionário interno da ONG.	Endereco
Usuario	Credenciais de login do colaborador.	Colaborador, Perfil
Perfil	Perfil de acesso de um usuário.	—
Especialidade	Especialidade odontológica.	—
DentistaEspecialidade	Ligação entre dentista e especialidade.	Dentista, Especialidade

8.4. Onde estão os métodos lógicos

Os quatro métodos lógicos apresentados na Seção 3 estão distribuídos entre três classes de entidade, conforme a tabela abaixo:

Classe	Método lógico
Triagem	definirElegibilidade e calcularPrioridade
Dentista	verificarDisponibilidade
Encaminhamento	encerrarEncaminhamento

9. Como Executar o Projeto

Esta seção reúne todas as instruções necessárias para colocar o backend Java do Projeto Nora em funcionamento. O objetivo é que o avaliador consiga rodar o projeto seguindo os passos na ordem apresentada.

9.1. Pré-requisitos

Antes de iniciar, é necessário ter os seguintes itens instalados e disponíveis:

- **Java 21** — a linguagem na qual o backend foi escrito.
- **Maven 3.9 ou superior** — ferramenta que compila e empacota o projeto. O projeto também inclui o Maven Wrapper, que dispensa a instalação separada do Maven.
- **Acesso ao banco de dados Oracle da FIAP** — o backend precisa se conectar a esse banco para funcionar.
- **Pelo menos um colaborador cadastrado no banco** — necessário para conseguir fazer login no sistema.

9.2. Preparação do banco de dados

O projeto inclui um script SQL completo que cria todas as tabelas e insere dados de exemplo. Esse script está no arquivo docs/database/setup_oracle.sql. Para preparar o banco:

1. Abrir uma ferramenta de acesso ao Oracle (por exemplo, o SQL Developer).
2. Conectar-se ao banco Oracle da FIAP com o usuário fornecido.
3. Abrir o arquivo setup_oracle.sql e executá-lo por completo.
4. Confirmar que as 15 tabelas foram criadas e que os dados de exemplo foram inseridos.

O script já cria as tabelas, insere registros de exemplo em todas elas e ainda inclui comandos de atualização, exclusão e relatórios SQL — úteis para validar que o banco está funcionando corretamente.

9.3. Configuração das variáveis de ambiente

Por segurança, os dados sensíveis (como usuário e senha do banco) não ficam escritos no código. Eles são informados por meio de variáveis de ambiente. O projeto inclui um arquivo de exemplo chamado .env.example, que deve ser copiado para um arquivo .env e preenchido com os valores reais. As variáveis necessárias são:

Variável	Obrigatória	Função
ORACLE_USER	Sim	Usuário do banco Oracle da FIAP.
ORACLE_PASS	Sim	Senha do banco Oracle da FIAP.
ORACLE_URL	Sim	Endereço de conexão (JDBC URL) do banco Oracle.
N8N_WEBHOOK_FOLLOWUP_URL	Não	Endereço para o disparo do follow-up automático. Se vazio, o follow-up roda em modo passivo.

Variável	Obrigatória	Função
ML_PREDICT_URL	Não	Endereço do serviço de predição de urgência. Se vazio, o backend usa o valor recebido na requisição.

Atenção à segurança

O arquivo `.env` com os valores reais nunca deve ser enviado ao repositório de código. Apenas o arquivo de exemplo (`.env.example`), sem dados reais, é versionado. Essa prática evita que senhas e credenciais fiquem expostas.

9.4. Executando o projeto localmente

Com o banco preparado e as variáveis configuradas, o projeto pode ser executado. Os comandos abaixo devem ser digitados no terminal, dentro da pasta do projeto.

Modo de desenvolvimento

O modo de desenvolvimento sobe a aplicação com recarregamento automático — ao alterar o código, ele é recarregado sem precisar reiniciar:

```
./mvnw quarkus:dev
```

Gerar a versão de produção

Este comando compila e empacota a aplicação em um arquivo executável:

```
./mvnw package -DskipTests
```

Executar a versão empacotada

Após o empacotamento, a aplicação pode ser executada com:

```
java -jar target/quarkus-app/quarkus-run.jar
```

9.5. Verificando se o projeto está no ar

Com a aplicação rodando, é possível confirmar que ela está funcionando acessando o endpoint de saúde. No terminal ou no navegador:

```
curl http://localhost:10000/q/health
```

Se a resposta indicar status “UP”, a aplicação está no ar e conectada ao banco. A partir desse momento, os demais endpoints da Seção 4 já podem ser utilizados.

9.6. Testando a API

Para testar os endpoints, recomenda-se uma ferramenta de testes de API, como o Postman ou o Insomnia. O projeto inclui uma coleção de testes pronta na pasta docs/api-collection, que pode ser importada nessas ferramentas. O roteiro de testes ponta a ponta está descrito no arquivo docs/testes-manuais.md.

O fluxo de teste recomendado segue esta ordem:

1. Fazer login no endpoint /auth/login e copiar o token recebido.
2. Usar esse token no cabeçalho Authorization de todas as demais requisições.
3. Cadastrar uma pessoa, criar uma triagem para ela e, em seguida, aprovar a triagem.
4. Conferir que o paciente e o encaminhamento foram criados automaticamente.
5. Consultar os endpoints de métricas para visualizar os indicadores.

9.7. Hospedagem (deploy)

O backend foi preparado para ser hospedado na plataforma Render. A configuração de deploy utiliza o comando de empacotamento para gerar a aplicação e o comando de execução para colocá-la no ar, com o endpoint de saúde sendo usado para monitoramento. Os detalhes completos do processo de deploy estão no arquivo docs/deploy-render.md do projeto.

10. Considerações Finais

10.1. Resumo do que foi entregue

O backend Java do Projeto Nora entrega uma API REST completa para a gestão da triagem odontológica da ONG Turma do Bem. O sistema cobre todo o ciclo do atendimento: do cadastro do adolescente até o acompanhamento do tratamento, passando pela triagem, pela aprovação e pelo encaminhamento a um dentista voluntário.

Ao longo deste documento foram apresentados: os objetivos e o escopo do projeto, suas funcionalidades, os quatro métodos lógicos exigidos com os respectivos prints de código, a tabela completa de endpoints, o fluxo crítico de aprovação, o modelo do banco de dados, o diagrama de classes e as instruções de execução.

10.2. Decisões técnicas de destaque

Algumas escolhas merecem registro por terem influenciado o resultado final:

- **Persistência com JDBC manual:** os comandos SQL são escritos à mão, o que torna visível e previsível cada operação no banco.
- **Regras de negócio nas entidades:** os métodos lógicos ficam junto dos dados que governam, facilitando localização e teste.
- **Aprovação como transação atômica:** o fluxo mais crítico do sistema é “tudo ou nada”, evitando estados inconsistentes no banco.
- **Organização em camadas:** cada parte do código tem uma responsabilidade clara, o que melhora a manutenção.
- **Credenciais fora do código:** dados sensíveis ficam em variáveis de ambiente, e não escritos no código-fonte.

10.3. Limitações conhecidas

Por se tratar de um projeto acadêmico, algumas simplificações foram adotadas de forma consciente e são registradas aqui com transparência:

- A seleção de dentista no fluxo de aprovação é feita internamente no banco, sem cálculo de distância geográfica — por isso o campo de distância fica vazio nos encaminhamentos automáticos.
- A autenticação utiliza um token simples; a comparação de senha é feita de forma direta, sem criptografia avançada.
- As listagens não possuem paginação — retornam todos os registros de uma vez.
- As métricas são calculadas a cada consulta, sem uso de cache.

10.4. Encerramento

O Projeto Nora demonstra, na prática, conceitos centrais do desenvolvimento de software: modelagem de dados, organização de código em camadas, criação de uma API REST, implementação de regras de negócio e tratamento de operações transacionais. Mais do que um exercício técnico, o projeto se propõe a apoiar uma causa real — facilitar o acesso de adolescentes em situação de vulnerabilidade a atendimento odontológico voluntário.

— *Fim da Documentação Técnica* —

Projeto Nora — Backend Java | FIAP — Análise e Desenvolvimento de Sistemas